MQTT events - what to remove, keep and add, per tag

Recommendations for the 13 event pages in `FXR_60-90_mqtt_api.json`, based on events captured live from two readers.

Document	FXR_60-90_mqtt_api.json - Zebra Fixed Reader MQTT API v1.0.0
Readers	10.233.48.36 FXR60 app 5.0.7 - 10.233.48.49 FXR90 app 5.0.4
Date	2026-09-07
Evidence	MQTT_EVENTS_OUTDATED.md and the .ndjson captures beside it
Live events captured	13 management - 350 tag data across 5 modes - 659 BLE

Two tag groups carry events:

```
Management Events  ->  Management-events   (8 pages)
Tag Data Events    ->  Tag-data-events     (5 pages)
```

Summary table

Page	Tag	Action
/async-events	Management-events	KEEP - regenerate the example
/heartbeat	Management-events	KEEP - regenerate the example
/gpi	Management-events	KEEP as-is [OK]
/gpo	Management-events	KEEP as-is [OK] verified
/error	Management-events	KEEP - replace "string"
/warning	Management-events	KEEP - replace "string"
/firmwareUpdateProgress	Management-events	KEEP - verify against a real update
/userapp_event	Management-events	[BUG] FIX - example is a copy of the heartbeat
/tagDataEvents	Tag-data-events	KEEP [OK] - add mode/metadata notes
/mode_tag_data_events	Tag-data-events	KEEP [OK] - keep as the shared base; see below
(missing) /simple_tag_data_events	Tag-data-events	? ADD - 23 events captured
(missing) /inventory_tag_data_events	Tag-data-events	? ADD - 54 events captured
(missing) /portal_tag_data_events	Tag-data-events	? ADD - GPI-triggered, distinct behaviour
(missing) /conveyor_tag_data_events	Tag-data-events	? ADD - 24 events captured
(missing) /custom_tag_data_events	Tag-data-events	? ADD - 21 events captured
/directionality_tag_data_events	Tag-data-events	REMOVE from FXR60/90 - unsupported
/zoneHistory	Tag-data-events	REMOVE from FXR60/90 - directionality only
/locationHistory	Tag-data-events	REMOVE from FXR60 - but FXR90 differs
(missing) /ble_data_events	Tag-data-events	? ADD - 659 events captured, 7 beacon types, entirely undocumented

---

REMOVE

1. /directionality_tag_data_events - the platform does not support it

Both readers report the capability as false, and the mode is rejected:

Reader	directionalitySupported
FXR60 5.0.7	false
FXR90 5.0.4	false

```
PUT /cloud/mode  {"type":"DIRECTIONALITY", ...}
FXR60 -> 422  "Directionality operating mode is Not Supported"
FXR90 -> 422  "Directionality operating mode is Not Supported"
```

The page says **"Applies To: FXR60 / FXR90"**. That is wrong for both.

Also remove `DIRECTIONALITY` from the `tagDataEvents type` enum, or the same impression is created a second time.

2. /zoneHistory - only exists inside directionality events

`zoneHistory` appears solely in a `TIMED_OUT` directionality event, and only when `report_zone_history` is enabled in directionality mode. With the mode unavailable, the payload is unreachable.

Its example is also two fields with an inconsistent casing that no other event uses:

```
{"timestamp": 1699540400724, "Zone": 2}
```

Capital `Zone`, and an epoch-millis `timestamp` where every other event uses ISO-8601. If the page is kept for other platforms, both should be checked against a real capture.

3. /locationHistory - remove for FXR60, but the models differ

Reader	tagLocationingSupported
FXR60 5.0.7	false
FXR90 5.0.4	true

This is the one case where the two models genuinely differ, so a combined "FXR60 / FXR90" label is wrong in both directions.

`locationHistory` is documented as arriving inside a directionality event, which neither model supports - so on that path it is unreachable on both. But the FXR90's `tagLocationingSupported: true` suggests locationing is available by some other route. **Not tested here**, and the flag alone does not prove it.

Recommend: state capabilities **per model**, and clarify whether locationing on the FXR90 has a delivery path that does not require directionality mode.

4. Nothing else

`/mode_tag_data_events` was initially assessed as a removable duplicate. **That was wrong** - see KEEP #7.

KEEP

5. /gpo - verified correct, no change needed

The only event page confirmed byte-accurate against live data:

```
doc  : {"pin": 1, "state": "LOW"}
live : {"pin": 1, "state": "HIGH"}
```

Same fields, same types, same casing. Only the state value differs, which is just what the pin happened to be.

6. /gpi - keep as-is

```
{"pin": 2, "state": "LOW"}
```

Structurally identical to `/gpo`, which is verified. Not directly observed - GPI needs a physical input change - but there is no reason to doubt it and no evidence against it.

7. /mode_tag_data_events - verified per-mode

Correction. This page was first assessed as a removable duplicate of `/tagDataEvents`. It is not. Its description is explicit:

> carries RFID tag read data reported during SIMPLE, INVENTORY, PORTAL, CONVEYOR, or CUSTOM operating modes

It documents the **`data` payload** for the non-directionality modes; `/tagDataEvents` documents the **envelope** that carries it. Two layers, not two versions.

All five modes were then captured live, and the field sets genuinely differ:

Mode	Events	type	data fields present
SIMPLE	23	SIMPLE	eventNum, format, idHex
INVENTORY	54	INVENTORY	+ antenna, peakRssi, reads
PORTAL	0	-	accepted, but emitted nothing in this setup
CONVEYOR	24	CONVEYOR	+ antenna
CUSTOM	21	CUSTOM	eventNum, format, idHex

Three observations worth adding to the page:

1. **`type` echoes the mode.** Each capture carried its own mode name - five distinct values, not the `"SIMPLE"` constant the example implies.
2. **The default field set is mode-dependent.** `INVENTORY` yields six fields unprompted; `SIMPLE` and `CUSTOM` yield three. The single example shows all 14 at once, which no mode produces by default.
3. **`PORTAL` produced no events.** Accepted with HTTP 200, but nothing arrived in a 12-second window on a single antenna with tags present. Its own trigger is a GPI or motion signal, so this is most likely a setup limitation rather than a fault - **not** established either way here.

Recommendation: keep this page as the shared base, and add one page per mode.

The five modes are documented today as a single sentence listing their names. But each emits a different default payload and has different trigger behaviour, so a consumer integrating PORTAL learns nothing from an example generated in SIMPLE. `/directionality_tag_data_events` already gets its own page - the other five modes deserve the same treatment.

See ADD #13 for what each page would contain.

8. /tagDataEvents - accurate; add two notes

Verified: the envelope matches and **11 of 14** `data` fields appeared live exactly as named. Keep the page. Two additions:

- **`type` carries the operating mode.** The example shows `"SIMPLE"`; live it was `"CUSTOM"`. Consumers must expect all five modes, not a constant.
- **Mark which `data` fields are conditional on `tagMetaData`.** A plain inventory returns **three** fields (`eventNum`, `format`, `idHex`); the example shows all 14 at once.

`XPC` was requested twice and never emitted across 228 events - its documented value is the placeholder `"string"`, so it is unverified on both sides.

FIX

9. /userapp_event - the example is the wrong event [BUG]

Its schema declares one property:

```
properties: ['event']
```

Its description says:

> A raw event string defined and formatted by the user application

But the example is a **verbatim copy of the `/heartbeat` example** - confirmed identical byte-for-byte, with top-level keys:

```
radio_control   reader_gateway   system   userapps
```

None of those is `event`. A copy-paste error: the page contradicts its own schema and its own description.

Replace it with something matching the schema, e.g.:

```
{"event": "door-controller: badge 4471 accepted"}
```

10. /heartbeat and /async-events - regenerate from a live reader

Both examples have drifted identically. Diffed against a live heartbeat:

Missing - the reader sends these:

Field	Note
reader_gateway.interfaceConnection	per-endpoint connection state + stats for control/data/management/managementEvent - the most useful monitoring content in the whole event
reader_gateway.dataPathStatistics	array of per-path counters
reader_gateway.numDataMessagesF	
system.GPI / system.GPO	full pin state maps
system.hostName, macAddress	
system.powerSource, powerNegotiation	e.g. PWR_BRICK, POE+

Stale - documented but not sent:

Field	Reality
reader_gateway.numDataMessagesRxed	moved into dataPathStatistics[]
reader_gateway.numDataMessagesTxed	moved into dataPathStatistics[]
reader_gateway.numDataMessagesDropped	moved into dataPathStatistics[]
reader_gateway.numDataMessagesRetained	moved into dataPathStatistics[]
system.flash.	flash is present but null
system.systemTime	reader sends systemtime - lowercase t

The four flat counters moving into an array is the breaking one: a client reading `reader_gateway.numDataMessagesRxed` finds nothing.

Regenerate from a capture rather than editing by hand - the structural changes are easy to miss.

11. /error and /warning - replace the placeholder

Both examples are:

```
{"message": "string"}
```

`"string"` is a type name, not a value. The reader had `numErrors: 7` in its heartbeat, so real errors exist and could be captured. Neither was observed during this window, so their true shape is unconfirmed - but a real message would tell an integrator whether to expect a code, a component prefix, or free text.

12. /firmwareUpdateProgress - verify during a real update

```
{"status": "started", "imageDownloadProgress": 0,
 "overallUpdateProgress": 0, "updateProgressDetails": null}
```

Plausible and complete, unlike the `"string"` placeholders. Not verifiable without running `set_os`, which was out of scope. Worth confirming the `status` enum and what `updateProgressDetails` contains when non-null.

ADD

13. ? One page per operating mode

There is **no page for any individual mode**. Confirmed by search: no path contains `simple`, `inventory`, `portal`, `conveyor` or `custom`. All five are covered by the one sentence in `/mode_tag_data_events`.

Yet `DIRECTIONALITY` - the one mode the hardware does *not* support - has a dedicated page, plus `zoneHistory` and `locationHistory`. The five modes that do work share one.

Each page should carry its captured payload:

/simple_tag_data_events - 23 events

```
{
  "data": {"eventNum": 5036, "format": "epc",
           "idHex": "e2806894000040017790e471"},
  "timestamp": "2026-09-07T12:50:36.194+0000",
  "type": "SIMPLE"
}
```

Three fields by default. No antenna, no RSSI unless requested via `tagMetaData`.

/inventory_tag_data_events - 54 events

```
{
  "data": {"antenna": 1, "eventNum": 5059, "format": "epc",
           "idHex": "e2806894000040017790e471",
           "peakRssi": -37, "reads": 1},
  "timestamp": "2026-09-07T12:50:55.513+0000",
  "type": "INVENTORY"
}
```

Six fields by default - the richest of the five. `antenna`, `peakRssi` and `reads` arrive without being requested. Also the highest event count, since `modeSpecificSettings.interval` controls the report cadence.

/portal_tag_data_events - GPI-triggered

The only mode whose event flow depends on an external signal:

```
{"type": "PORTAL", "antennas": [1], "transmitPower": 30,
 "modeSpecificSettings": {"startTrigger": {"port": 1, "signal": "HIGH"},
                          "stopInterval": 5}}
```

Accepted with HTTP 200, and **zero events** over a 20-second window with tags in range - because the GPI trigger never fired. That is correct behaviour, not a fault, and it is exactly what a dedicated page needs to say: *no tag data is published until the configured `startTrigger` asserts.*

Without that, an integrator sees an accepted config and a silent stream, and has no way to tell whether the mode is broken. This is the strongest case of the five for its own page.

/conveyor_tag_data_events - 24 events

```
{
  "data": {"antenna": 1, "eventNum": 5113, "format": "epc",
           "idHex": "e2806894000040017790e471"},
  "timestamp": "2026-09-07T12:51:29.949+0000",
  "type": "CONVEYOR"
}
```

Four fields - `antenna` by default, but no `peakRssi` or `reads`.

/custom_tag_data_events - 21 events

```
{
  "data": {"eventNum": 5137, "format": "epc",
           "idHex": "e2806894000040017790e471"},
  "timestamp": "2026-09-07T12:51:55.441+0000",
  "type": "CUSTOM"
}
```

Three fields - same default as SIMPLE, but every field is configurable through `tagMetaData`, `accesses`, `filter` and `reportFilter`. The mode where the payload varies most by configuration, so its page should show the metadata-rich form alongside the bare one.

Default field summary

Mode	Default data fields	Count
SIMPLE	eventNum, format, idHex	3
CUSTOM	eventNum, format, idHex	3
CONVEYOR	+ antenna	4
INVENTORY	+ antenna, peakRssi, reads	6
PORTAL	(none until the GPI trigger asserts)	0

All five samples are in `mode_samples.json`; full captures in `mode_SIMPLE.ndjson`, `mode_INVENTORY.ndjson`, `mode_PORTAL.ndjson`, `mode_CONVEYOR.ndjson`, `mode_CUSTOM.ndjson`.

14. ? BLE data events - completely undocumented

The file has **no BLE event page at all**. Under the `Ble` tag there are only two command pages, `get_bleConfig` and `set_bleConfig`. Searching the whole document:

Term	Occurrences
BLE_DATA	0
beaconType	0
ManufacturerData	0
iBeacon / altBeacon / eddystone	22 / 11 / 11 - all inside bleConfig filters, none in an event

So the configuration surface is documented and the resulting event stream is not.

659 BLE events were captured in one 18-second scan, all `type: BLE_DATA`, covering **seven** beacon types:

beaconType	Count
GENERIC_BLE	308
IBEACON	99
ALTBEACON	90
EDDYSTONE_TLM	54
EDDYSTONE_URL	36
EDDYSTONE_UID	36
EDDYSTONE_EID	36

A real event, verbatim:


```
{
  "data": {
    "Adapter": "/org/bluez/hci0",
    "Address": "CF:C7:E6:45:DE:F7",
    "AddressType": "random",
    "Alias": "CF-C7-E6-45-DE-F7",
    "Blocked": false,
    "Connected": false,
    "LegacyPairing": false,
    "Paired": false,
    "RSSI": -66,
    "ServicesResolved": false,
    "Trusted": false,
    "UUIDs": ["0000feaa-0000-1000-8000-00805f9b34fb"],
    "beaconType": "EDDYSTONE_TLM",
    "eddystone": {"advPduCount": 27432, "batteryVoltage": 3300,
                  "frameType": "TLM", "temperature": -5.0,
                  "timeSincePowerOn": 918381, "version": 0}
  },
  "eventNum": 1900,
  "timestamp": "2026-09-07T12:53:07.484+00:00",
  "type": "BLE_DATA"
}
```


The `data` object carries **18 fields**, and a protocol-specific sub-object keyed by `beaconType`:

beaconType	Sub-object	Fields
IBEACON	iBeacon	uuid, major, minor, txPower
ALTBEACON	altBeacon	beaconId, mfgId, major, minor, refRssi
EDDYSTONE_URL	eddystone	frameType, txPower, url
EDDYSTONE_UID	eddystone	frameType, txPower, namespace, instance
EDDYSTONE_EID	eddystone	frameType, txPower, ephemeralId
EDDYSTONE_TLM	eddystone	frameType, version, batteryVoltage, temperature, advPduCount, timeSincePowerOn
GENERIC_BLE	(none)	base fields only

Two things a page would need to state. First, `BLE_DATA` shares the `tagDataEvents` envelope and arrives on the **same data endpoint** as RFID tag reads - a consumer must switch on `type` to separate them. Second, the field casing is **capitalised** (`Address`, `RSSI`, `Alias`, `AddressType`), unlike every RFID field, and unlike the lowercase form `bleConfig` requires in its `generic` filter. That mismatch has already caused a rejected payload elsewhere in this testing.

One sample per beacon type is in `ble_samples_by_beacontype.json`; the full capture is `mode_BLE.ndjson`.

15. An example per type on /async-events

`/async-events` documents seven `type` values and ships **one** example (`heartbeat`), despite its own note:

> Always check the `type` field before parsing `data`. The `data` shape is different for each event type.

The individual pages exist, but a consumer reading the envelope page sees only one shape. Add the envelope form of each - they are small:

```
{"component": "RG",
 "data": {"pin": 1, "state": "HIGH"},
 "eventNum": 9,
 "timestamp": "2026-09-07T12:21:53.635+0000",
 "type": "gpo"}
```

That is a real captured event.

16. State that antennas maps are device-dependent

The heartbeat example shows **13** antennas (`"1"`...`"13"`). The FXR60 reports **6**, the FXR90 reports **6**. Same for `numTagReadsPerAntenna`. The 13-entry example reads as a fixed shape.

17. Note the two time formats

One event contains both:

Field	Format
timestamp	2026-09-07T12:21:53.635+0000 - ISO-8601
system.systemtime	07/09/2026 12:22 - DD/MM/YYYY HH:MM

Worth stating explicitly so parsers are written correctly.

What was and was not verified

Event	Verified	How
gpo	[OK]	12 live events, matched field-for-field
heartbeat	[OK]	1 live event, diffed against both pages
tagDataEvents	[OK]	228 live events across 3 configurations
directionality	[OK] unsupported	capability flag + 422 on both models
gpi	[-]	needs a physical input change
error, warning	[-]	needs a fault condition
firmwareUpdateProgress	[-]	needs an OS update
userapp_event	[-] payload	needs an installed app - but the copy-paste error is provable from the file alone
mode_tag_data_events	[OK]	122 events across SIMPLE / INVENTORY / CONVEYOR / CUSTOM
PORTAL mode	[-]	accepted, but emitted no events in this setup
BLE_DATA	[OK]	659 events, 7 beacon types
zoneHistory, locationHistory	[-]	unreachable without directionality

Five payload shapes remain unconfirmed. The recommendations above distinguish *"verified wrong"* from *"unverified"* - nothing is marked for removal on the basis of not having observed it, only where the reader's own capability flags or error responses prove it.